From GitHub to production

You receive the code. Fast Deploy manages its release. Google Cloud runs the application.

STAGING

A separate place to try the app before real users depend on it.

PRODUCTION

The live application that real users access.

THE EIGHT STEPS

01	**Receive the repository**	You + developer
02	**Review the deployment settings**	You + Fast Deploy
03	**Prepare the cloud resources**	Fast Deploy + Terraform
04	**Test and build the app package**	GitHub Actions
05	**Run and check staging**	Cloud Run + you
06	**Approve the production release**	An authorized reviewer
07	**Move requests to the new version**	The release workflow
08	**Serve users and plan the next update**	Cloud Run + Fast Deploy

Fast Deploy is the coordinator. GitHub Actions performs the build and release work. Cloud Run hosts the running app.

Code and deployment settings

A repository is a project folder stored in GitHub. You have an app repository and a platform repository.

1 Receive a complete application

The developer supplies code, dependency files, tests and startup instructions. Required data must be included or delivered separately with clear instructions.

2 Review how Fast Deploy will run it

`platform.yaml` describes settings such as the port, health check, memory, access and needed services. Fast Deploy can draft this file for review.

OPTION A

Settings beside the app

BetterCare repository

- Application code
- Dependency files and tests
- **platform.yaml**
- App deployment workflows

Fast Deploy reads the code and settings from this repo.

OPTION B

Settings kept by the platform

BetterCare repository

Application code, dependencies, tests

Fast Deploy repository

`apps/bettercare/platform.yaml`

The app repo stays unchanged for this configuration step.

The path above is an example. In Option B, the platform file also identifies the source repository and commit. A person reviews and merges that file. The two file formats are different; let Fast Deploy generate the correct one. For a private app, Option B also requires the platform repository to be private.

What the developer must still provide

Fast Deploy cannot invent missing app data. The supplied BetterCare copy lacks `data/index/chunks.json`, which its server needs to start, and `data/site/`, which its replica website needs.

Code: onboarding.py, contract.py, models.py; BetterCare server/corpus.mjs and site/server.mjs.

Prepare, test and package

The cloud needs a place to run the app. The app needs a repeatable, runnable package.

3 Prepare the cloud resources

First, an operator connects Fast Deploy to GitHub and Google Cloud and grants the required permissions. Terraform then creates each app's declared cloud resources through reviewed infrastructure changes.

Cloud Run

Runs the application

Artifact Registry

Stores app packages

Secret Manager

Supplies private keys

A database or file bucket is created when the application declares that need. The developer still writes the code that uses it. Staging and production use separate Google Cloud projects.

4 Build the exact version selected for release

Fast Deploy identifies a specific Git commit: a saved version of the code. It checks the settings and starts the GitHub Actions workflow.

Check

Tests + required security checks

→

Build

App code + dependencies

→

Store

Versioned package in Artifact Registry

This package is called an **image**. A Dockerfile is the recipe used to build it. Fast Deploy can generate a basic recipe; app-specific preparation may require a custom Dockerfile.

What stops the process here?

A failed required test, a blocking security finding or a build error stops this release. Build work runs without cloud deployment credentials; a separate job gets permission to deploy.

Code: onboarding.py, scaffold.py, controller.py, templates/deploy-gcp.yml and infra/gcp/.

Try it, then approve it

Staging is the rehearsal. Production uses the same tested image with its own environment settings.

5 Start the app in staging

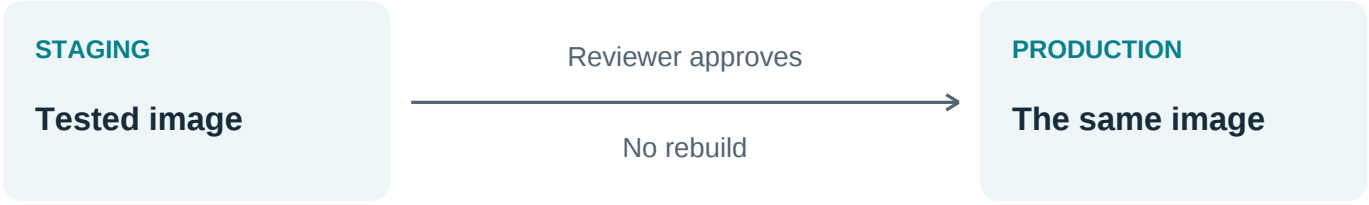
- Cloud Run starts a new app version. Normal service requests still go to the previous version.
- The platform checks that the new version starts and responds at its health address. BetterCare uses `/api/health`.
- After the check passes, staging requests move to the new version. Its logs are watched for five minutes by default.
- You test the actual features: opening the page, searching, chatting and saving history when enabled.

A health check is a basic readiness check

A successful health response does not prove that search results, AI answers or saved history are correct. Those need application tests.

6 Approve the move to production

You request promotion of a successful, signed staging release. An authorized reviewer approves the run in GitHub. The platform verifies and copies the same image into production.



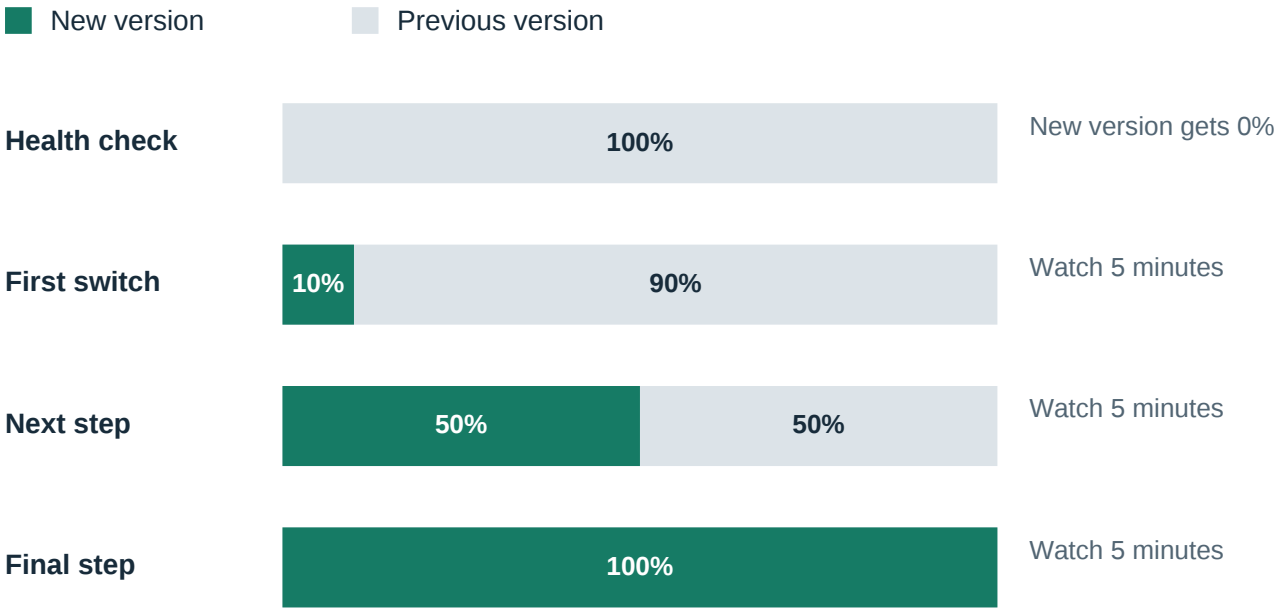
Same image, separate environment

Production resources, data and secrets must be ready. Promotion copies the image; it does not transfer the contents of a staging database or file bucket. A configured database update command runs against production before new-version traffic begins.

Code: controller.py promote(); release.py rollout(), deploy(), require_secrets() and migrate().

Move traffic in small steps

Traffic means requests sent to the app. These percentages are not download progress or a fixed list of users.



After the initial approval, the steps are automatic. The workflow follows fixed rules. With these defaults, all requests reach the new version about 10 minutes after the first switch; the final watch ends about 15 minutes after that first switch. Preparation time is extra.

If a release rule fails

The workflow stops advancing and attempts to restore the traffic split that existed before this release. It records whether that restoration succeeded.

The rules watch for a crash, more than 5% server errors with at least 20 requests, or a slow-response measure (p95) above 3 times the previous version's. The speed comparison needs at least 20 requests for each version. Health-check requests are excluded.

p95 is the time within which 95% of measured requests completed.

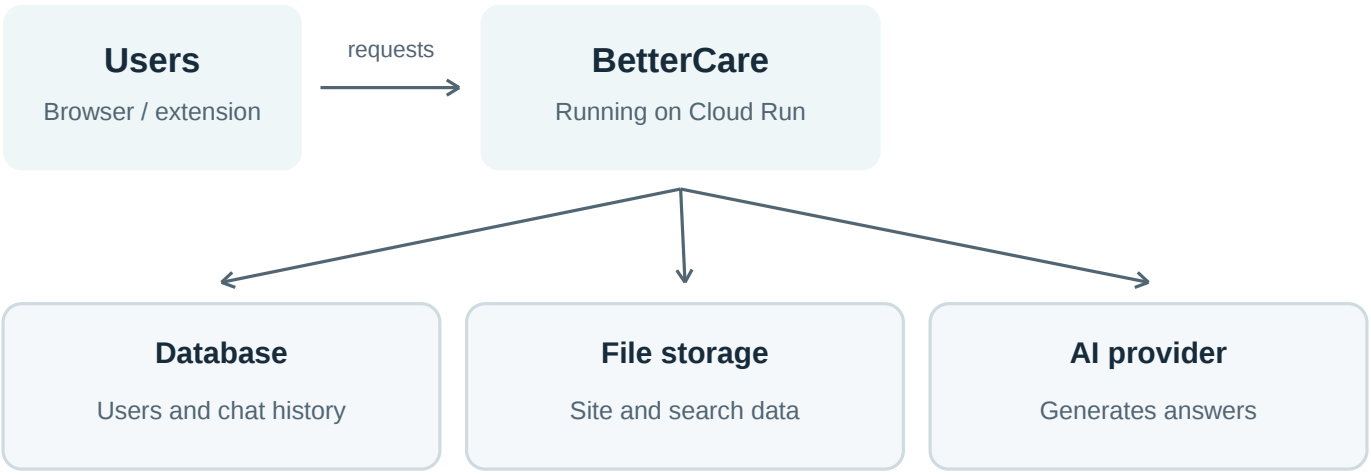
Three limits to understand

Too little traffic or unreadable logs: the current code can continue with an inconclusive or unobserved result.
Rollback: restores traffic, not database changes.
First-ever release: the previous version is a temporary placeholder, not an earlier BetterCare release. Finish the launch before inviting users.

The live app and the next update

Once released, the application serves users on Cloud Run. Fast Deploy manages the release process around it.

8 Let the application serve users



These connections exist only when the app implements and configures them. Fast Deploy is not in the path of each user request. Its own database records deployment activity, not BetterCare chat history.

What happens when the developer changes the code?

- **Automatic staging is off by default.** You can request a release. If auto-deploy is enabled, Fast Deploy can release new commits to staging automatically.
- **Production still needs approval.** A successful staging release does not automatically replace production.
- **The release watch is time-limited.** The five-minute checks on the previous page are part of that release, not a promise of permanent automatic rollback.

Before handing the live app to users

Confirm that the intended users can sign in, the actual features work, required data is present and any history that must survive is stored persistently. For BetterCare, its current demo identity handling and local SQLite storage need attention before normal multi-user use.

Code: config.py, controller.py, release.py; BetterCare server/index.mjs, store.mjs and config.mjs.

Developer repository checklist

A general checklist for Python and Node.js web applications. These files make the app understandable and repeatable to deploy.

Prepare the application files below. Documentation and example settings are handoff recommendations; the exact deployment gates depend on the chosen setup.

FILE OR FOLDER	WHAT THE DEVELOPER SHOULD INCLUDE
Application code	All source files, the server entry file and any assets the app reads while running.
Node.js dependencies	`package.json` with working start and test scripts, plus a matching `package-lock.json`.
Python dependencies	`requirements.txt` for runtime packages. Include `requirements-dev.txt` for tests; a `requirements.lock` is recommended for repeatable installs.
`test/` **or** `tests/`	Tests and their dependencies. Tests can be disabled in reviewed settings, but then the release has no unit-test gate.
`README.md`	How to install, start and test; runtime version; port and health path; required settings, data and external services.
`.env.example`	Names of settings with safe placeholder values and explanations. Real passwords and API keys stay outside Git.
`.gitignore`	Exclude real .env files, credentials, installed packages and local caches. Do not accidentally omit required app assets.

Only when needed: include database migration scripts, build scripts and instructions or download locations for required data and model files. Large data can be delivered separately; its exact location must be documented.

Useful additions: LICENSE, SECURITY.md and a CI workflow that runs checks on code changes. These are recommended repository hygiene, not universal deployment blockers.

Example repository structures

Use one layout for the language of the app. Folder and entry-file names below are examples, not required names.

NODE.JS EXAMPLE

```
my-app/
  package.json
  package-lock.json
  .nvmrc
  .env.example
  .gitignore
  README.md
  server/
    index.mjs
  test/
    health.test.mjs
```

PYTHON EXAMPLE

```
my-app/
  requirements.txt
  requirements.lock
  requirements-dev.txt
  .python-version
  .env.example
  .gitignore
  README.md
  app/
    main.py
  tests/
    test_health.py
```

The version files (.nvmrc and .python-version) state the tested runtime version. Keep installed dependencies such as node_modules/ and .venv/ out of Git.

Files Fast Deploy can prepare

A basic `Dockerfile`, `.dockerignore`, `platform.yaml` and deployment workflows. With app-owned workflows, the reviewed app configuration lives in the app repo. With platform-managed import, it can live in the platform repo instead (page 2).

Check these details before handoff

Special build: include a custom Dockerfile if extra preparation is needed. The default Node.js image does not run npm run build.

Branch and folder: app-owned workflows use main. Platform-managed import can use the source default branch. Identify the service folder if the repo has several apps.

Existing Python projects: platform-managed import also accepts supported pyproject.toml and lockfile layouts. A working project need not be reorganized just to match this example.

Code: onboarding.py, reader.py, scaffold.py and models.py. Do not commit real secrets or local package folders.